

Executive Architecture & Authorization Fundamentals

Enterprise Policy Interceptor Architecture for Agentic AI

VOLUME COVERAGE

Covers enterprise authorization landscape, externalized authorization theory, PEP/PDP/PAP/PIP architecture, separation of authentication from authorization, Zero Trust principles, and the rationale for policy-as-code in Agentic AI deployments.

Executive Summary

This document series presents a comprehensive, implementation-oriented architecture guide for building an enterprise-grade authorization and request interception framework for Agentic AI systems deployed on AWS. The framework externalizes authorization decisions from application code and enforces them through policy-as-code using AWS Cedar (Amazon Verified Permissions) and Open Policy Agent (OPA/Rego).

Modern enterprises — particularly those in regulated industries such as banking, healthcare, and government — face an unprecedented challenge: AI agents now act autonomously on behalf of users, invoking tools, querying knowledge stores, accessing databases, and calling enterprise APIs. Each of these actions must be subject to the same rigorous authorization controls as any human user interaction, yet traditional authorization models were never designed for non-human actors operating at machine speed.

✓ **BEST PRACTICE**

Every request made by an agent, tool, workflow, API, MCP server, RAG system, memory system, and downstream application must be policy-evaluated before execution. This is not optional in regulated environments — it is a compliance requirement under PCI DSS, SOC 2, NIST 800-53, and the EU AI Act.

Target Environment

Dimension	Detail
Cloud Platform	Amazon Web Services (AWS)
Agent Runtime	ECS / EKS / Lambda / Amazon Bedrock AgentCore
Identity Provider	Microsoft Entra ID (primary) + Legacy ADFS federation
Authentication	SAML 2.0 / OIDC — JWT tokens with enterprise claims
API Protection	OAuth 2.0 / OIDC — enterprise application APIs
Tool Surface	MCP Servers exposing discrete tools
Authorization Engine	AWS Cedar (Amazon Verified Permissions) + OPA/Rego
Architecture Pattern	Zero Trust — default deny, policy-evaluated every request
Deployment Model	Multi-tenant, multi-agent workflows
Regulatory Context	Banking: PCI DSS, SOC 2, NIST 800-53, DORA, EU AI Act

Document Series Overview

This implementation guide is organized into five volumes, each addressing a distinct layer of the enterprise authorization stack:

Volume	Title	Covers
1	Executive Architecture & Fundamentals	Authorization theory, PEP/PDP/PAP/PIP, Zero Trust, landscape
2	Identity, Claims & Policy Design	ADFS/Entra federation, claims normalization, Cedar & Rego design

Volume	Title	Covers
3	Agent & Tool Authorization	Agent lifecycle, tool permissions, MCP security, context-aware authz
4	RAG, Memory & Data Authorization	Document-level authz, vector filtering, memory protection, tenant isolation
5	AWS Implementation & Governance	Reference architecture, CI/CD, compliance mapping, production checklist

1. Why Externalized Authorization?

1.1 The Problem with Embedded Authorization

The dominant pattern in enterprise software for decades has been embedding authorization logic directly within application code — checking roles in service methods, validating permissions in controllers, or hardcoding group membership checks. This approach creates a series of structural problems that compound as systems grow in complexity:

- **Authorization Logic Drift:** Each service re-implements authorization logic independently, leading to inconsistency, duplication, and divergence over time.
- **Policy Invisibility:** Authorization rules are buried in code. Security teams cannot audit, review, or modify them without developer involvement and a full deployment cycle.
- **No Separation of Concerns:** Business logic and security policy are entangled. A change to either requires touching the same codebase.
- **Inability to Audit:** When a security incident occurs, there is no central record of what decisions were made, when, and why.
- **Agent Incompatibility:** AI agents operate as non-human principals, often at machine speed with parallelism. Embedded authorization cannot scale or reason about agent context.
- **Compliance Failure:** Regulatory frameworks require demonstrable, auditable access controls. Code-embedded authorization cannot satisfy auditors without exhaustive code reviews.

X ANTI-PATTERN

Anti-Pattern: `if (user.groups.contains('Finance_Approvers')) { approvePayment(); }` — This pattern embeds identity store details, business rules, and authorization logic in a single statement. It cannot be audited, versioned, tested, or changed without a code deployment.

1.2 The Externalized Authorization Model

Externalized authorization separates the authorization decision from the application code. The application delegates the authorization question to a dedicated policy engine and acts on the result. This is the foundation of all modern enterprise authorization frameworks.

The flow transforms from a tightly coupled embedded check to a structured policy evaluation pipeline:

BEFORE (Embedded): Request → Application Code → IF (role check) THEN execute ELSE reject AFTER (Externalized): Request → PEP (Intercept) → PDP (Evaluate) → Decision → PEP (Enforce) → Service

1.3 The Three Pillars of Modern Enterprise Authorization

Modern architectures enforce a strict separation between three distinct concerns:

- **Authentication** — *Who are you?*
Verifies the identity of a principal (user, service, agent). Produces a verified identity token (JWT). Managed by Entra ID, ADFS, or an IdP. Authorization does not begin until identity is established.
- **Authorization** — *What are you allowed to do?*
Determines whether the authenticated principal may perform a specific action on a specific resource in a given context. Managed by the policy engine (Cedar/OPA). Authorization is context-dependent and evaluated at request time.
- **Business Logic** — *How should it be done?*
The actual execution of the business operation after authorization is confirmed. Business services never make authorization decisions — they trust the PEP enforcement layer.

2. Authorization Architecture: PEP, PDP, PAP, PIP

The XACML-originated model of Policy Enforcement Point, Policy Decision Point, Policy Administration Point, and Policy Information Point remains the definitive reference architecture for externalized authorization. All modern systems — Cedar, OPA, OpenFGA, Permit.io — are implementations of this model.

Component	Role	AWS Implementation	Key Characteristics
PEP (Enforcement)	Intercepts every request; enforces the decision returned by the PDP. Never executes without a positive decision.	API Gateway Lambda Authorizer, Envoy sidecar, App Mesh, ALB Listener Rules, application middleware	Must be in the critical path. Zero-trust: default deny if PDP is unreachable. Logs every decision.
PDP (Decision)	Evaluates the request against applicable policies and returns Allow/Deny with optional obligations.	Amazon Verified Permissions (Cedar), OPA sidecar/central server, Styra DAS	Stateless evaluation. Deterministic. Must be low-latency (<5ms P99). Horizontally scalable.
PAP (Administration)	Where policies are authored, reviewed, tested, versioned, and deployed.	AVP Policy Store, OPA Bundle Server (S3), Git repository, Styra DAS console, CI/CD pipeline	GitOps-driven. Policy-as-code. Version controlled. PR-based review. Automated testing.
PIP (Information)	Provides additional attributes about the subject, resource, or environment required for policy evaluation.	DynamoDB attribute store, ElastiCache, Directory Lookup Lambda, SCIM endpoint, STS, Secrets Manager	Enriches the authorization context. Cached aggressively. Authoritative source of truth for attributes.

2.1 PEP Implementation Patterns

The PEP is the most architecturally sensitive component. It must be in the critical path of every request, yet it must add minimal latency. Enterprises implement PEPs at multiple layers:

Pattern	Technology	Latency	Best For
API Gateway Authorizer	AWS Lambda Authorizer	10–50ms	REST APIs, external-facing services

Pattern	Technology	Latency	Best For
Envoy/Service Mesh Filter	Istio + OPA, AWS App Mesh	1–5ms	Internal microservices, East-West traffic
Application Middleware	FastAPI middleware, Express middleware, Spring Filter	0.5–2ms	Fine-grained business-context authorization
gRPC Interceptor	gRPC UnaryInterceptor / StreamInterceptor	0.5–2ms	gRPC service-to-service calls
Sidecar Container	OPA sidecar on ECS/EKS pod	1–3ms	Language-agnostic, containerized services
Reverse Proxy	NGINX + OPA, Kong Gateway	2–10ms	Legacy application fronting
Event Stream Filter	EventBridge Rule + Lambda	Async	Event-driven agent workflows

3. Authorization Engine Landscape

Enterprises have multiple policy engine options. The choice depends on the authorization model required, the existing technology stack, the scale of policy management, and the specific use case (infrastructure vs. application vs. agent authorization).

Engine	Language	Model	Strength	Weakness	Best For
AWS Cedar (Verified Permissions)	Cedar	RBAC+ABAC+ReBAC	Type-safe, provably correct, AWS-native, formal verification	AWS-only, no infrastructure policy	Application & agent authorization on AWS
OPA / Rego	Rego (Datalog-like)	General purpose	Universal, Kubernetes-native, WASM, rich ecosystem	Rego learning curve, no built-in schema validation	Infrastructure, K8s admission, cross-platform
XACML	XML Policy	ABAC	Standards-based, mature, rich attribute model	Verbose XML, complex, slow adoption	Legacy enterprise, government standards compliance
OpenFGA	OpenFGA DSL	ReBAC (Zanzibar)	Relationship-first, Google Zanzibar-inspired, Google-maintained	Relatively new, limited ABAC	Fine-grained relationship-based permissions
Zanzibar / SpiceDB	Zed language	ReBAC	Proven at Google scale, consistent global authorization	Operational complexity, self-managed	Global-scale relationship authorization
Permit.io	Policy-as-code + UI	RBAC+ABAC+ReBAC	Developer-friendly, multi-model, managed	SaaS dependency, cost at scale	Teams needing rapid policy iteration
Styra DAS	Rego + UI	OPA management	Enterprise OPA management, GitOps, audit logs	OPA dependency, cost	Enterprise OPA fleet management

Engine	Language	Model	Strength	Weakness	Best For
Oso	Polar language	RBAC+ReBAC	Embedded in app, simple syntax, developer-first	Less ecosystem, limited at enterprise scale	Developer-embedded authorization
AuthZed / SpiceDB	Zed language	ReBAC	Zanzibar-compatible, strongly consistent	Relationship model complexity	Zanzibar-style fine-grained permissions

✓ BEST PRACTICE

Enterprise Recommendation: For AWS-hosted Agentic AI with Entra ID, the optimal architecture combines AWS Cedar (Amazon Verified Permissions) for application and agent authorization with OPA/Rego for infrastructure and Kubernetes policy. These are complementary, not competing engines. Volume 2 details the hybrid architecture.

4. Zero Trust Architecture for Agentic AI

Zero Trust mandates that no request — regardless of its origin, network position, or prior authentication — is trusted by default. Every request must be explicitly verified, authorized, and continuously validated. For Agentic AI, this principle is not merely desirable; it is essential.

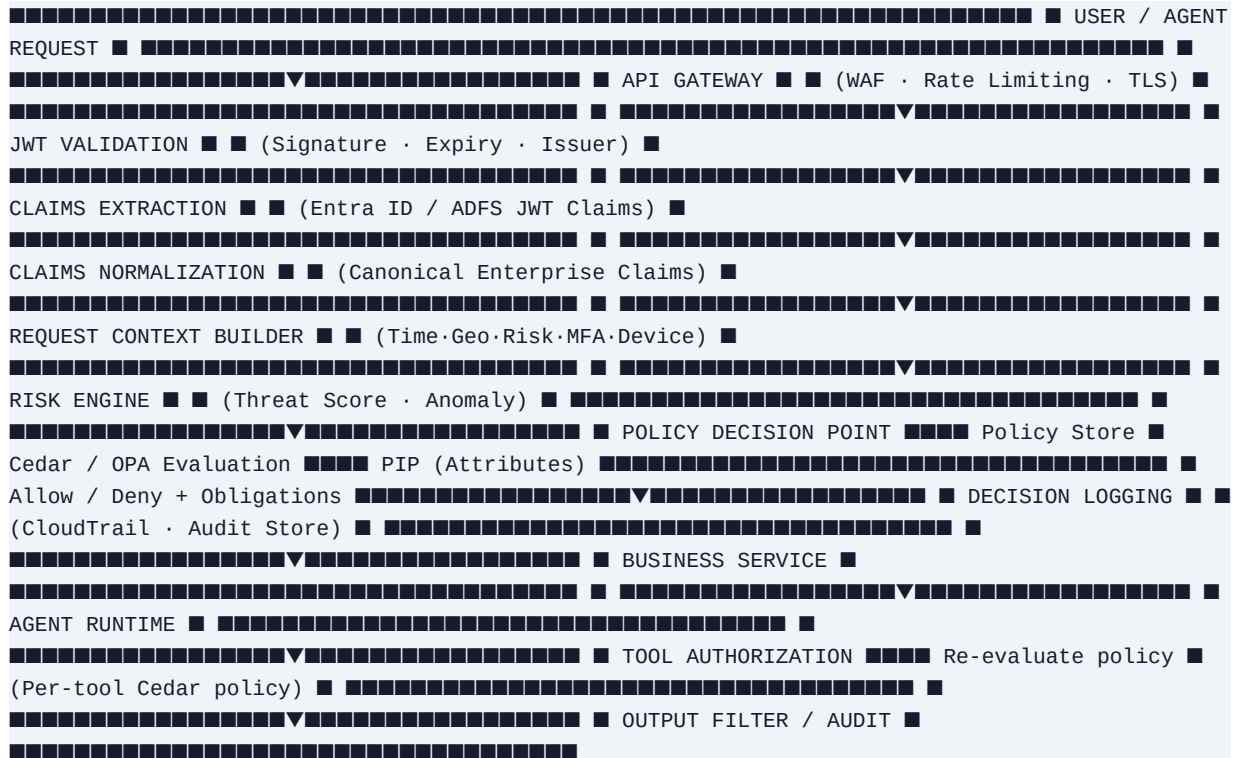
Zero Trust Principle	Implementation for Agentic AI
Never Trust, Always Verify	Every tool call, API request, and memory access by an agent is independently authorized — even within a single agent workflow.
Least Privilege Access	Agents receive the minimum permissions required for their specific task. Permissions are not inherited from the invoking user wholesale.
Assume Breach	Authorization decisions are logged, monitored, and anomaly-detected. Agents cannot escalate privileges even if a component is compromised.
Explicit Verification	Every authorization decision includes the principal identity, resource, action, context, and policy reason.
Micro-segmentation	Agent-to-agent communication, agent-to-tool calls, and agent-to-data access are separately authorized. No transitive trust.
Continuous Validation	Authorization is re-evaluated at each action boundary. A session-level token is insufficient for tool-level authorization.

5. The Request Interceptor Pattern

The request interceptor is the architectural foundation that transforms embedded authorization into externalized policy evaluation. It is the PEP in practice.

5.1 Complete Request Pipeline

The following pipeline represents the authoritative request flow for an Agentic AI system. Every layer is mandatory for regulated enterprise deployments:



NOTE

Critical Design Principle: Authorization is evaluated at EVERY boundary crossing — not just at the API gateway. An agent invoking a tool, a tool querying a database, and the agent reading from memory are all separate authorization events requiring independent policy evaluation.

6. Threat Model: STRIDE for Agentic AI Authorization

Applying the STRIDE threat model to the authorization layer identifies the key threats that the policy interceptor architecture must mitigate:

Threat	Example in Agentic AI	Mitigation
Spoofing	Agent impersonates another user or agent to gain elevated permissions	JWT signature validation, agent identity claims, mutual TLS between agents
Tampering	Agent modifies request context to claim false attributes (e.g., elevated role)	Signed context tokens, immutable audit logs, context validation at PDP
Repudiation	Agent denies having invoked a tool or accessed data	Comprehensive decision logging to CloudTrail, signed audit records, non-repudiation via KMS
Info Disclosure	Agent retrieves documents or memory belonging to another tenant	Tenant isolation in Cedar policies, document-level authorization, output filtering
Denial of Service	Excessive tool calls overwhelm PDP evaluation capacity	PDP caching, rate limiting at PEP, circuit breakers, auto-scaling PDP fleet
Elevation of Privilege	Agent accumulates permissions across workflow steps (confused deputy)	Per-step authorization, no permission accumulation, explicit obligation enforcement

7. Critical Anti-Patterns to Avoid

The following anti-patterns represent the most common authorization failures observed in enterprise AI deployments. Each is a compliance risk and a security vulnerability:

✗ ANTI-PATTERN

Authorization Logic in Business Code: Security policy cannot be audited, versioned, or changed without code deployment. Violates separation of concerns.

✗ ANTI-PATTERN

Policies Referencing Entra Groups Directly: Cedar policies that check if principal belongs to 'Payments_Admins' AD group create tight coupling. Group membership changes break authorization without policy changes.

✗ ANTI-PATTERN

Tool Execution Without Authorization: Agent tools that execute SQL, call APIs, or write to storage without per-invocation authorization create uncontrolled blast radius.

✗ ANTI-PATTERN

Overly Large JWTs: Attempting to encode all authorization attributes in the JWT token creates tokens that exceed HTTP header limits and are impossible to revoke mid-session.

✗ ANTI-PATTERN

Missing Default-Deny Semantics: Any authorization model that allows actions not explicitly denied (instead of denying all not explicitly allowed) is fundamentally insecure.

✗ ANTI-PATTERN

No Policy Versioning: Policies that cannot be rolled back when they cause authorization failures are operational liabilities in production.

✗ ANTI-PATTERN

Policy Evaluation After Execution: Checking authorization after the fact violates Zero Trust and cannot prevent harm.

✗ ANTI-PATTERN

No Decision Logging: Without a record of every authorization decision, breach investigation is impossible and regulatory compliance cannot be demonstrated.

✗ ANTI-PATTERN

Lack of Audit Trails: Fine-grained audit records (who accessed what, when, authorized by which policy) are mandatory for banking regulation compliance.

✗ ANTI-PATTERN

Hard-Coded Claims: Embedding specific group names or role strings in policy code rather than using canonical claim mappings creates a maintenance nightmare.

8. Regulatory Compliance Mapping

The authorization architecture described in this series maps directly to controls required by major regulatory frameworks applicable to banking and financial services:

Framework	Relevant Controls	How This Architecture Satisfies Them
NIST 800-53	AC-2, AC-3, AC-6, AU-2, AU-9, CM-3	Externalized authorization (AC-3), least privilege (AC-6), comprehensive audit logging (AU-2/AU-9), policy-as-code change management (CM-3)
NIST AI RMF	GOVERN 1.1, MAP 2.3, MEASURE 2.6, MANAGE 1.3	Policy-governed AI actions (GOVERN), risk-contextualized authorization (MAP), measurable policy compliance (MEASURE), incident response via policy rollback (MANAGE)
PCI DSS v4.0	Req 7, Req 8, Req 10	Need-to-know access control (Req 7), unique identity for agents (Req 8), audit logs for all access to cardholder data (Req 10)
SOC 2 Type II	CC6.1, CC6.2, CC6.3, CC7.2	Logical access controls (CC6.1), authentication management (CC6.2), authorization reviews (CC6.3), monitoring of access controls (CC7.2)
ISO 27001:2022	A.5.15, A.5.18, A.8.2, A.8.3	Access control policy (A.5.15), access rights management (A.5.18), privileged access (A.8.2), information access restriction (A.8.3)
DORA	Art. 9, Art. 10	ICT risk management (Art. 9), ICT-related incident management (Art. 10) — policy audit trails are mandatory evidence
EU AI Act	Art. 9, Art. 13, Art. 17	Risk management system (Art. 9), transparency & logging (Art. 13), quality management for high-risk AI (Art. 17)

Next Steps

Volume 2 of this series covers **Identity, Claims & Policy Design** — including Entra ID and ADFS federation patterns, JWT claims normalization, canonical enterprise claim models, Cedar policy design patterns, and Rego policy architecture.